

Artificial Intelligence 2nd set ex.

1.A.

University of Piraeus is organizing an event where 7 historical figures will talk and their speeches must be divided into 4 periods $\Pi 1$, $\Pi 2$, $\Pi 3$, $\Pi 4$, though we are facing limitation problems. The speakers are Alan Turing (T), Ada Lovelace (L), Niels Bohr(B), Marie Curie (C), Socrates(Σ), Pythagoras (Π) and Isaac Newton(N).

Limitations:

- Alan Turing must speak in the first period only.
- The Physics Lovers want to hear Niels Bohr, Isaac Newton.
- Math enthusiasts want to hear Lovelace, Pythagoras, and Newton
- Philosophy lovers want to hear Socrates, Pythagoras.
- The visitors from Patras want to hear only Lovelace and Curie.
- Visitors from Cyprus want to hear only Turing, Lovelace, and Newton.
- I want to hear Curie and Pythagoras

We can think of 2 ways of expressing that problem:

- We can represent each Speaker using a different variable. Then our Variables will be T, L, B, C, Σ , Π , N, and the domain will include the 4 different periods of time that each speaker can talk: $D = \{\Pi 1, \Pi 2, \Pi 3, \Pi 4\}$, and limitations are:
 - i. **$T = \Pi 1$** (Turing can only talk in the 1st period)
 - ii. **$B \neq N$** (Bohr and Newton cannot speak in the same period)
 - iii. **$L \neq \Pi$, $L \neq N$, $\Pi \neq N$** (Lovelace, Pythagoras and Newton cannot speak in the same period)
 - iv. **$\Sigma \neq \Pi$** (Socrates and Pythagoras cannot speak in the same period)

- We can represent each period as a different variable. Then our variables will be $\Pi_1, \Pi_2, \Pi_3, \Pi_4$, and the domain will include all scientists $D = \{T, L, B, C, \Sigma, \Pi, N\}$, and the limitations are:

- 1.B.

[illegible]

To avoid collision also we 'll note for each variable the collisions it might have:

	T	L	B	C	Σ	Π	N
	L, N	Π, N, C, T	N	Π, L	Π	Σ, L, N, C	T, L, Π, B

Forward Checking:

Whenever a speaker is assigned to a period, their assigned value is removed from the domains of all other speakers that are constrained not to overlap with them. This pruning process ensures that conflicts are avoided early and prevents invalid assignments from propagating further into the solution.

Step	T	L	B	C	Σ	Π	N
	{Π1}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}
T= Π1	{Π1}	{Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π2, Π3, Π4}
L= Π2	{Π1}	{Π2}	{Π1, Π2, Π3, Π4}	{Π1, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π3, Π4}	{Π3, Π4}
B=Π1	{Π1}	{Π2}	{Π1}	{Π1, Π3, Π4}	{Π1, Π2, Π3, Π4}	{Π1, Π3, Π4}	{Π3, Π4}
C=Π1	{Π1}	{Π2}	{Π1}	{Π1}	{Π1, Π2, Π3, Π4}	{Π3, Π4}	{Π3, Π4}
Σ= Π1	{Π1}	{Π2}	{Π1}	{Π1}	{Π1}	{Π3, Π4}	{Π3, Π4}
Π=Π3	{Π1}	{Π2}	{Π1}	{Π1}	{Π1}	{Π3}	{Π4}
N=Π4	{Π1}	{Π2}	{Π1}	{Π1}	{Π1}	{Π3}	{Π4}

Step				
T	Π1			
L	Π1	Π2	Π3	Π4
B	Π1	Π2	Π3	Π4
C	Π1	Π2	Π3	Π4
Σ	Π1	Π2	Π3	Π4
Π	Π1	Π2	Π3	Π4
N	Π1	Π2	Π3	Π4

1.Γ.

With the new flexibility in Alan Turing's schedule, where he can now speak in any period $\{\Pi1, \Pi2, \Pi3, \Pi4\}$, the order in which the speakers are assigned can be optimized to reduce conflicts and enhance efficiency during backjumping with forward checking. The new order prioritizes the most constrained speakers first, as their limited options are more likely to cause issues later if not resolved early. The suggested order is **Niels Bohr (B)** and **Isaac Newton (N)** (due to the Physics Enthusiasts' constraints), followed by **Ada Lovelace (L)** and **Marie Curie (C)** (due to overlapping group preferences), then **Pythagoras (P)** and **Socrates (Σ)** (for the Philosophy and Mathematics groups), and finally **Alan Turing (T)**, since his new flexibility allows him to adapt to the remaining available periods. This approach minimizes conflicts in the domains and reduces the need for backtracking, making the scheduling process more efficient.

2.

Step 1: CNF

$(\neg p \vee \neg q) \Rightarrow (s \vee t)$ $\neg(\neg p \vee \neg q) \vee (s \vee t)$ Using De Morgan's laws: $(p \wedge q) \vee (s \vee t)$ This expands to: 1: $p \vee s \vee t$ 2: $q \vee s \vee t$	$s \Rightarrow p$ 3. $\neg s \vee p$	$\neg s \Rightarrow \neg t$ 4. $s \vee \neg t$	$\neg(p \vee q)$ $\neg p \wedge \neg q$ This gives two clauses: 5: $\neg p$ 6: $\neg q$
--	---	---	---

2.A

Resolution is a proof technique used in propositional and first-order logic to derive conclusions by iteratively combining clauses to eliminate variables. Starting with a set of premises expressed in CNF, pairs of clauses are selected where a variable and its negation appear. By resolving them (removing the variable), new clauses are generated. The process continues until either the empty clause (indicating a contradiction) is derived, proving the initial formula is unsatisfiable, or no further resolution is possible. It is a key method for automated reasoning.

Resolution 1: Clause 1 and Clause 5 - Clause 1: $(p \vee s \vee t)$ - Clause 5: $(\neg p)$ - Resolving: Remove p and $\neg p \rightarrow (s \vee t)$ New Clause: - Clause 7: $(s \vee t)$
Resolution 2: Clause 2 and Clause 6 - Clause 2: $(q \vee s \vee t)$ - Clause 6: $(\neg q)$ - Resolving: Remove q and $\neg q \rightarrow (s \vee t)$ New Clause: - Clause 8: $(s \vee t)$ (same as Clause 7, so it doesn't add new information)

Resolution 3: Clause 7 and Clause 4 • Clause 7: $(s \vee t)$ • Clause 4: $(s \vee \neg t)$ • Resolving: Combine the two clauses, focusing on t and $\neg t \rightarrow (s)$ New Clause: • Clause 9: (s)
Resolution 4: Clause 9 and Clause 3 • Clause 9: (s) • Clause 3: $(\neg s \vee p)$ • Resolving: Remove s and $\neg s \rightarrow (p)$ New Clause: • Clause 10: (p)
Resolution 5: Clause 10 and Clause 5 • Clause 10: (p) • Clause 5: $(\neg p)$ • Resolving: Contradiction!

2.B

The **DPLL algorithm** is a method for solving the satisfiability problem of propositional logic formulas in **Conjunctive Normal Form (CNF)**. It systematically assigns truth values to variables to determine if the formula can be satisfied. The algorithm uses three key techniques: **unit propagation**, which simplifies the formula by assigning truth values to variables in single-literal clauses and propagating the effects; **pure literal elimination**, which assigns truth values to literals that appear with only one polarity (always positive or always negative) across all clauses; and **backtracking**, which explores alternative assignments if a conflict arises. When a partial assignment leads to an unsatisfiable state, it backtracks to previous choices and tries other possibilities. The process terminates when all clauses are satisfied (returning satisfiable) or when all possible assignments have been exhausted (returning unsatisfiable). DPLL is efficient and forms the foundation for modern SAT solvers.

From $\neg p$ and $\neg q$: • Clause 1: $(p \vee s \vee t) \rightarrow (\text{false} \vee s \vee t) \rightarrow (s \vee t)$. • Clause 2: $(q \vee s \vee t) \rightarrow (\text{false} \vee s \vee t) \rightarrow (s \vee t)$. • Result: The updated clauses are: ○ $(s \vee t)$ ○ $(s \vee t)$ ○ $(\neg s \vee \neg t)$
Combine $(s \vee t)$ and $(\neg s \vee \neg t)$: 1. If $s = \text{true}$, then $\neg s = \text{false}$, forcing $t = \text{false}$. 2. If $t = \text{true}$, then $\neg t = \text{false}$, forcing $s = \text{false}$. At this point, there is no conflict.
Check $s \Rightarrow (p \wedge \neg t)$: • If $s = \text{true}$, then: ○ $p = \text{true}$ and $\neg t = \text{true}$. ○ But this contradicts the initial clause $\neg p$.

3.A

Depth-First Search with Backtracking

The Depth-First Search (DFS) with backtracking explores all possible assignments for the variables, ensuring that every assignment satisfies the given constraints. The process proceeds as follows:

1. Starting Point: We begin with the variable s, as Constraint 3 explicitly requires $s=0$. This eliminates $s=1$ from the domain. The assignment $s=0$ is chosen.
2. Applying Constraint 2: The constraint $\neg s \Rightarrow (\neg r \wedge \neg p)$ now activates. Since $s=0$, we must have $r=0$ and $p=0$. These assignments are added to our current solution.
3. Exploring q Under Constraint 1: Next, we consider the remaining variable q, as it is part of Constraint 1 $(p \wedge q) \vee r$. With $p=0$ and $r=0$, the constraint simplifies to

$(0 \wedge q) \vee 0 = 0$, regardless of whether $q=0$ or $q=1$. Thus, Constraint 1 cannot be satisfied for any value of q .
4. Conflict and Conclusion: As no valid assignment for q satisfies Constraint 1, we backtrack to explore alternative assignments. However, all possible branches ultimately lead to conflicts. Therefore, the algorithm concludes that no solution exists for the problem.
<pre> graph TD Start((Start)) --> s0((s=0)) s0 --> pr0((p=0, r=0)) pr0 --> q0((q=0)) pr0 --> q1((q=1)) q0 --> Conf0((Conflict)) q1 --> Conf1((Conflict)) </pre>

3.B

Arc Consistency (AC-3) Algorithm

The **AC-3 algorithm** ensures that all constraints are consistent by removing values from the variable domains that cannot satisfy any constraint.

1. Initial Constraints: ○ $s=0$ (from Constraint 3), ○ $s=0 \Rightarrow (r=0 \wedge p=0)$ (from Constraint 2), ○ $(p \wedge q) \vee r=1$ (from Constraint 1).
2. Reduce Domains: ○ From $s=0$, remove $s=1$ from its domain: $s:\{0\}$ ○ From $s=0 \Rightarrow (r=0 \wedge p=0)$, remove $r=1$ and $p=1$: $r:\{0\}$, $p:\{0\}$ ○ Domains now: $p:\{0\}$, $q:\{0,1\}$, $r:\{0\}$, $s:\{0\}$

3. Check Constraint 1 $((p \wedge q) \vee r = 1)$:

- Substituting $p=0$ and $r=0$, the constraint becomes: $(0 \wedge q) \vee 0 = 0$ (0 and q) or 0 = 0 $(0 \wedge q) \vee 0 = 0$
- This cannot be satisfied because the expression evaluates to 000.

4. Conclusion:

- The constraints are inconsistent, so no solution exists.

